

第四章 DSP系统设计入门

4.1 TMS320C28x的 C语言编程

4.2 TMS320F28027最小系统简介

4.3 实验套件应用扩展示例

4.4 集成开发环境CCS

C2000的寻址方式:

一、直接寻址

二、堆栈寻址

三、间接寻址

四、寄存器寻址

五、数据程序IO空间立即寻址

六、程序空间间接寻址

七、字节寻址

4.1 TMS320C28x的 C语言编程

●概述：C语言优势

- ✓ C语言是高级语言，相对于汇编易学易用；
- ✓ C语言的编程效率极高，易于调试；
- ✓ C++运行效率不如C，且面向对象编程占用更多资源；
- ✓ Java同C++，且Java运行依赖于Java虚拟机；

汇编语言依赖于计算机硬件，程序的可读性和可移植性较差。

高级语言具有很好的可移植性，但是难以实现汇编语言的某些功能（如对内存地址的操作、位操作等）。

C 语言作为一种高级语言，既可以访问物理地址又可以进行位操作，能直接对硬件进行操作，适合用于DSP

所以，嵌入式编程多用C语言，或C语言与汇编混合编程。F20827亦是如此。

●4.1.1 基于TM320C28x的C语言

(1) 数据类型

数据类型	长度	描述	最小值	最大值
char, signed char	16bits	ASCII	-32768	32767
unsigned char	16bits	ASCII	0	65535
short, signed short	16bits	2s补码	-32768	32767
unsigned short	16bits	二进制	0	65535
int, signed int	16bits	2s补码	-32768	32767
unsigned int	16bits	二进制	0	65535
long, signed long	32bits	2s补码	-2147483648	2147483647
unsigned long	32bits	二进制	0	4294967295
enum	16bits	2s补码	-32768	32767
float	32bits	IEEE 32-bit	1.19209290e-38	3.4028235e+38
double	32bits	IEEE 32-bit	1.19209290e-38	3.4028235e+38
long double	32bits	IEEE 32-bit	1.19209290e-38	3.4028235e+38
pointers	16bits	二进制	0	0xFFFF
far pointers	22bits	二进制	0	0x3FFFFFF

(2) 重要标识符

const

表示变量内容为常数，不会改变。

使用：const+数据类型+变量名。

举例：const char tab[1024]={显示数据};

volatile

用于声明存储器或外设寄存器，以此来说明所定义的变量是“可变的”，是可以被DSP的其他硬件修改的，而不仅仅是由C程序本身修改。

使用：volatile+数据类型+变量名。

举例：volatile struct SYS_CTRL_REGS SysCtrlRegs;

cregister

允许采用高级语言直接访问控制寄存器。

使用：cregister+数据类型+变量名。

举例：extern cregister volatile unsigned int IFR;
extern cregister volatile unsigned int IER;

interrupt

用于说明函数是中断函数，这样编译器会自动为中断函数产生**保护现场和恢复现场**所需执行的操作。

使用：interrupt+void+中断函数名(void);

举例：interrupt void XINT1_ISR(void);

(3) C语言中嵌入汇编语言

TMS320C28x编译器允许在C程序中嵌入汇编指令，通过下面声明实现：

```
asm(“assembler text”);
```

其中assembler text指汇编代码。asm指令一般用来处理C/C++语句较难实现的硬件操作。

```
举例： #define EINT    asm(“ clrc INTM”)  
        #define DINT  asm(“ setc INTM”)
```


(4) 存储器模型

TMS320C28x将存储器分成**程序和数据**两个线性块。

程序存储器包含可**执行代码**，**初始化数据**和**开关量**。

数据存储器包含**外部变量**、**静态变量**和**系统堆栈**。

编译器产生可重定位的代码，**运行链接器**将代码和数据分配到适当的存储器空间

编译器产生的代码和数据块称为section（段），section分为**初始化段**和**非初始化段**两种类型。

编译器中的段名

已初始化段

名称	说明	链接内存位置
.text	代码	闪存
.cinit	用于全局和静态变量的初始化值	闪存
.econst	常数 (例如: 整数常数 $k = 3$;)	闪存
.switch	分支语句表	闪存
.pinit	全局构造子表 (C++)	闪存

未初始化段

名称	说明	链接内存位置
.ebss	全局和静态变量	RAM
.stack	堆栈空间	低地址的 64Kw RAM
.esysmem	内存分配函数 far malloc	RAM

注: 由于仿真器可用于 RAM 加载, 因此在开发过程中可将已初始化段链接至 RAM

C语言中的代码段

全局变量 (.ebss)

初值 (.cinit)

```
int x = 2;
```

```
int y = 7;
```

```
void main(void)
```

```
{
```

```
    long z;
```

```
    z = x + y;
```

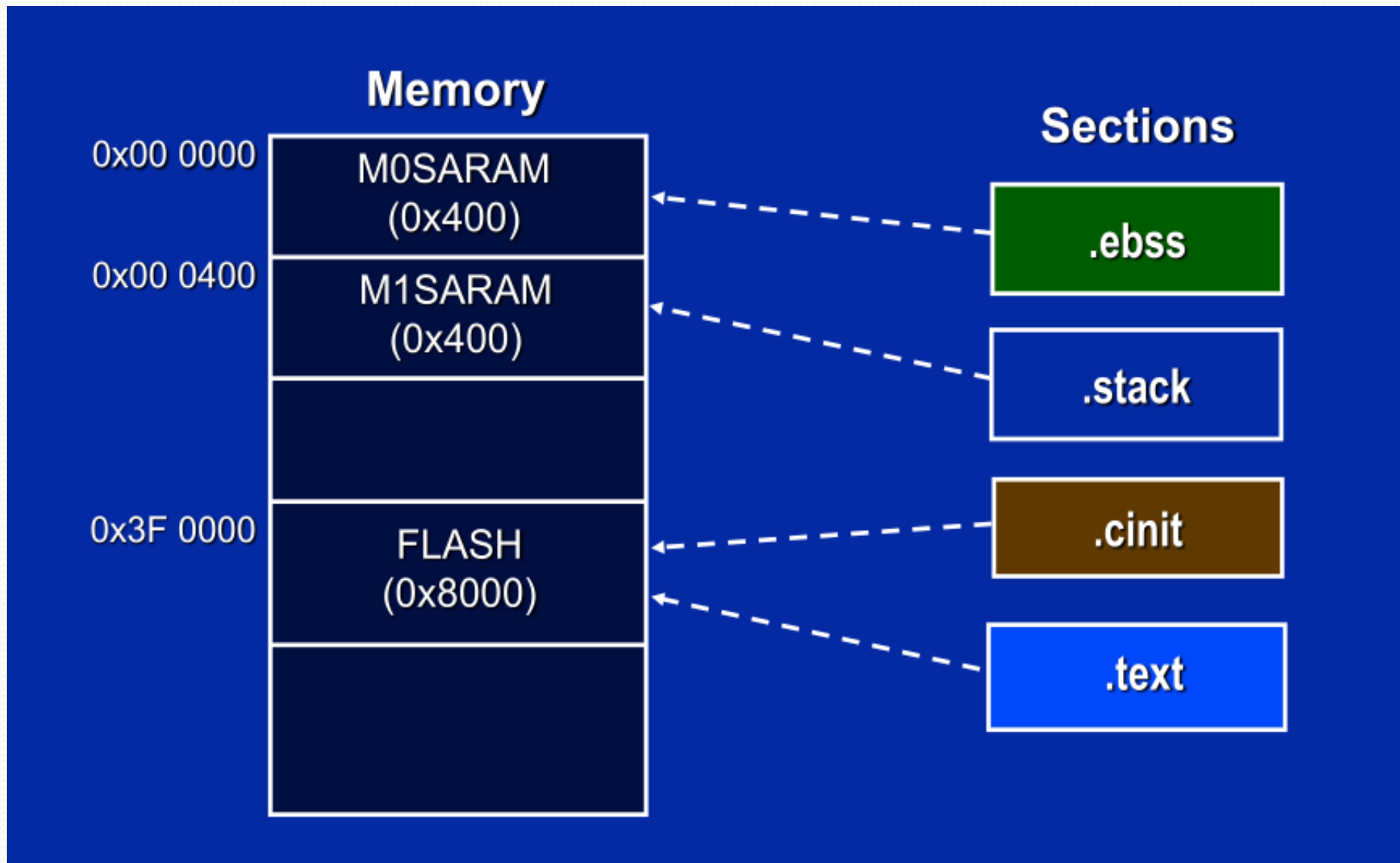
```
}
```

局部变量 (.stack)

代码 (.text)

- ◆ 所有代码均由被称为“段”的不同部分组成
- ◆ 所有的默认段名均以“.”作为开始
- ◆ 编译器具有用于 已初始化 和 未初始化 代码段的默认段名

存储器中不同段的分配



(5) pragma伪指令

通过pragma伪指令告诉编译器**如何对待特定**的函数、对象或代码段。

C28x C/C++编译器支持如下形式的pragma伪指令：

```
CODE_SECTION(func, "section name");
```

```
DATA_SECTION(symbol, "section name");
```

CODE_SECTION

用于为函数func在一个名为section name的段中指定空间。

C语言的标准用法是：

```
#pragma CODE_SECTION(func, "section name");
```

举例：

```
#pragma CODE_SECTION(sum, "codeA");
```

```
int sum(int a, int b);
```

```
void main(void)
```

```
{    int a=1,b=2,c;  
      c=sum(a, b); }
```

```
int sum(int a, int b)
```

```
{    return(a+ b);}
```


DATA_SECTION

用于为变量symbol在一个名为section name的段中指定空间。

C语言的标准用法是：

```
#pragma DATA_SECTION(symbol, "section name");
```

举例：

```
#pragma DATA_SECTION(SysCtrlRegs, "dataA");  
volatile struct SYS_CTRL_REGS SysCtrlRegs;
```


●4.1.2 链接命令（CMD）文件

DSP片上存储空间分配包含两个部分：

片上外设存储空间分配

用户自定义数据及程序的存储空间分配

分别对应于两个链接命令文件，以后缀.cmd结尾，简称CMD文件。

CMD文件的两大功能：指示存储空间、分配段到存储空间

CMD文件主要采用MEMORY和SECTIONS两条伪指令

MEMORY伪指令

作用：指示存储空间。

MEMORY

{

PAGE 0: name 0[(attr)]: origin= constant, length= constant

PAGE n: name n[(attr)]: origin= constant, length= constant

}

PAGE: 用来指示存储空间的关键字。n的最大值为255。

name: 某一属性和地址范围的存储空间名称。长度可以是1-8个字符。在**同一页内名称不能相同**，**不同页内名称可以相同**。

attr: 用来规定存储空间的属性，共有四种属性：R-只读，W-只写，X-该空间包含可执行代码，I-该空间可以被初始化。

origin: 用来定义存储空间起始地址的关键字。

length: 用来定义存储空间长度的关键字。

SECTIONS伪指令

作用：分配段到存储空间，指定段的实际硬件地址空间

SECTIONS

{

name 0:> 存储空间名称, PAGE=页数

name n:> 存储空间名称, PAGE=页数

}

name: 编译器输出段的名称。

存储空间名称：采用MEMORY伪指令指示的存储器空间名称。

PAGE: 前面存储器空间名称对应的存储器页。

0x00 0000

0x00 0040

0x00 0400

0x00 0800

0x00 0D00

0x00 0E00

0x00 2000

0x00 6000

0x00 7000

0x00 8000

0x00 9000

0x3D 7800

0x3D 7C00

0x3D 7C80

0x3D 7CC0

Data Space	Prog Space
<i>M0 Vector RAM (Enabled if VMAP = 0)</i>	
M0 SARAM (1K x 16, 0-Wait)	
M1 SARAM (1K x 16, 0-Wait)	
Peripheral Frame 0	Reserved
PIE Vector - RAM (256 x 16) (Enabled if VMAP = 1, ENPIE = 1)	
Peripheral Frame 0	
Reserved	
Peripheral Frame 1 (4K x 16, Protected)	Reserved
Peripheral Frame 2 (4K x 16, Protected)	
L0 SARAM (4K x 16) (0-Wait, Secure Zone + ECSL, Dual Mapped)	
Reserved	
User OTP (1K x 16, Secure Zone + ECSL)	
Reserved	
Calibration Data	
Get_mode function	

0x3D 7CE0	Reserved
0x3D 7E80	Calibration Data
0x3D 7EB0	Reserved
0x3D 7FFF	PARTID
0x3D 8000	Reserved
0x3F 0000	FLASH (32K x 16, 4 Sectors, Secure Zone + ECSL)
0x3F 7FF8	128-Bit Password
0x3F 8000	L0 SARAM (4K x 16) (0-Wait, Secure Zone + ECSL, Dual Mapped)
0x3F 9000	Reserved
0x3F E000	Boot ROM (8K x 16, 0-Wait)
0x3F FFC0	<i>Vector (32 Vectors, Enabled if VMAP = 1)</i>

1) 用户自定义数据及程序的存储空间分配CMD

```
MEMORY{  
    PAGE 0:  /* Program Memory */  
        FLASH: origin = 0x3F0000, length = 0x8000  
    PAGE 1:  /* Data Memory */  
        M0SARAM: origin = 0x000000, length = 0x400  
        M1SARAM: origin = 0x000400, length = 0x400  
}  
SECTIONS  
{  
    .text:> FLASH          PAGE = 0  
    .ebss:> M0SARAM        PAGE = 1  
    .cinit:> FLASH          PAGE = 0  
    .stack:> M1SARAM       PAGE = 1  
}
```


举例:

MEMORY

```
{
PAGE 0:  /* Program Memory */
    PRAML0    : origin = 0x008000, length = 0x000800    /* on-chip RAM block L0 */
    OTP        : origin = 0x3D7800, length = 0x000400    /* on-chip OTP */
    FLASHD     : origin = 0x3F0000, length = 0x002000    /* on-chip FLASH */
    FLASHC     : origin = 0x3F2000, length = 0x002000    /* on-chip FLASH */
    FLASHA     : origin = 0x3F6000, length = 0x001F80    /* on-chip FLASH */
    BEGIN      : origin = 0x3F7FF6, length = 0x000002
    RESET      : origin = 0x3FFFC0, length = 0x000002    /* part of boot ROM */
    VECTORS     : origin = 0x3FFFC2, length = 0x00003E    /* part of boot ROM */

PAGE 1 :  /* Data Memory */

    BOOT_RSVD  : origin = 0x000000, length = 0x000050    /* Part of M0, BOOT rom will use
this for stack */
    RAMM0      : origin = 0x000050, length = 0x0003B0    /* on-chip RAM block M0 */
    RAMM1      : origin = 0x000400, length = 0x000400    /* on-chip RAM block M1 */
    DRAML0     : origin = 0x008800, length = 0x000800    /* on-chip RAM block L0 */
    FLASHB     : origin = 0x3F4000, length = 0x002000    /* on-chip FLASH */
}
```

SECTIONS

```
{  
    /* Allocate program areas: */  
    .cinit      : > FLASHA    PAGE = 0  
    .pinit      : > FLASHA,    PAGE = 0  
    .text       : > FLASHA    PAGE = 0  
    codestart    : > BEGIN     PAGE = 0  
  
    /* Allocate uninitialized data sections: */  
    .stack       : > RAMM0     PAGE = 1  
    .ebss        : > DRAML0    PAGE = 1  
    .esysmem     : > DRAML0    PAGE = 1  
  
    /* Initialized sections go in Flash */  
    .econst      : > FLASHA    PAGE = 0  
    .switch      : > FLASHA    PAGE = 0  
  
    .reset       : > RESET,    PAGE = 0, TYPE = DSECT  
    vectors      : > VECTORS   PAGE = 0, TYPE = DSECT  
  
}
```


强调：

用户自定义数据及程序的存储空间由用户根据软件功能合理分配，具有一定的**灵活性**。

在实际应用中可通过此CMD文件将部分**实时性要求比较高的程序重新装载到RAM中进行运行**

CMD文件也可以将程序代码链接到Flash或者RAM

三种不同的CMD文件：

正常模式：程序在FLASH或者OTP中，数据在RAM中

完全RAM模式：程序和数据都在RAM中，不使用FLASH和OTP

用户内存分配FLASH模式：在RAM中划一段区域用作程序空间，其他用作数据空间；DSP启动后某一段在FLASH中的代码被装载到RAM程序空间，且该段代码在零等待的RAM中执行。

2) 片上外设存储空间分配

片上外设存储空间分配由**DSP**型号即硬件结构决定，**一般不进行修改！！！！**

有BIOS和nonBIOS两种方式供用户，**前者不包含PieVectTable，而后者则包含了PieVectTable。**

一般使用nonBIOS方式：**DSP2802x_nonBIOS.cmd**

DSP2802x_nonBIOS.cmd

在SECTIONS中：

伪指令**GROUP**：表示几个输出段在存储空间是连续分配的，在例子中EmuKeyVar、EmuBModeVar、FlashCallbackVar和FlashScalingVar等在存储数据存储空间是按顺序连续分配的。

伪指令**UNION**：表示在程序运行过程中多个段占用相同的存储空间，在例子中PieVectTableFile和GROUP定义的段占用相同的数据空间。

●4. 1. 3 C28x的C语言编程风格

传统编程风格

```
#define ADCTRL1 (volatile unsigned int *)0x00007100
...
void main(void)
{
    *ADCTRL1 = 0x1234; //write entire register
    *ADCTRL1 |= 0x4000; //enable ADC module
}
```

优点

- 简单、快速且易于键入
- 变量名称与寄存器名称精确匹配 (易于记忆)

缺点

- 需要生成专门的掩码以操控个别位
- 不能很容易地在观察窗口中显示位字段
- 在许多场合中将生成效率低下的代码

新编程风格

```
void main(void)
{
    AdcRegs.ADCTRL1.all = 0x1234;
                                //write entire register
    AdcRegs.ADCTRL1.bit.ADCENABLE = 1;
                                //enable ADC module
}
```


优点

- 易于操控个别位。
- 易于在调试窗口观察寄存器位的值
- 可生成最高效的代码 (利用 C28x)

缺点

- 结构名称会难以记忆 (编辑器自动完成功能)
- 需要键入的内容更多 (编辑器自动完成功能)

结构体定义规则

- **F2802x** 头文件定义了：
- 所有的外设结构体
- 所有的寄存器名称
- 所有的位字段名称
- 所有的寄存器地址

PeripheralName.RegisterName.all

// 访问整个16位或 32位寄存器

PeripheralName.RegisterName.half.LSW

// 访问 32 位寄存器的 16 个低位

PeripheralName.RegisterName.half.MSW

// 访问 32 位寄存器的 16 个高位

PeripheralName.RegisterName.bit.FieldName

// 访问寄存器的指定位字段

注:

[1] “PeripheralName” 由 TI 指定并载于 F2802x 标准头文件。

它们由大写字母和小写字母组合成 (如: CpuTimer0Regs)。

[2] “RegisterName” 与数据表中使用的名称相同, 它们始终用大写字母表示 (如: TCR、TIM、TPR, ...)

[3] “FieldName” 与数据表中使用的名称相同, 它们始终用大写字母表示 (如: POL、TOG、TSS, ...)

F2802x 头文件包

(<http://www.ti.com> , 文献编号: SPRC832)

- ☐ 包含了使用结构体法所需的一切资源
- ☐ 定义了所有的外设寄存器位和寄存器地址

- ☐ 头文件包包括:
 - ☐ \DSP2802x_headers\include ☐ .h files
 - ☐ \DSP2802x_headers\cmd ☐ linker .cmd files
 - ☐ \DSP2802x_headers\gel ☐ .gel files for CCS
 - ☐ \DSP2802x_examples ☐ 2802x examples
 - ☐ \doc ☐ documentation

C 语言源文件 (例如:Adc.c)

```
#include "DSP2802x_Device.h"
Void InitAdc(void)
{
    /* Reset the ADC module */
    AdcRegs.ADCTRL1.bit.RESET = 1;
    /* configure the ADC register */
    AdcRegs.ADCTRL1.all = 0x00E4;
};
```

外设结构体 头(.h) 文件

```
// ADC Individual Register Bit Definitions:
struct ADCCTL1_BITS { // bits description
    Uint16 TEMPCONV:1; // 0 Temperature sensor connection
    Uint16 VREFLOCONV:1; // 1 VSSA connection
    Uint16 INTPULSEPOS:1; // 2 INT pulse generation control
    Uint16 ADCREFSEL:1; // 3 Internal/external reference select
    Uint16 rsvd1:1; // 4 reserved
    Uint16 ADCREFPWD:1; // 5 Reference buffers powerdown
    Uint16 ADCBGPWD:1; // 6 ADC bandgap powerdown
    Uint16 ADCPWDN:1; // 7 ADC powerdown
    Uint16 ADCBSYCHN:5; // 12:8 ADC busy on a channel
    Uint16 ADCBSY:1; // 13 ADC busy signal
    Uint16 ADCENABLE:1; // 14 ADC enable
    Uint16 RESET:1; // 15 ADC master reset
};
// Allow access to the bit fields or entire register:
union ADCCTL1_REG {
    Uint16 all;
    struct ADCCTL1_BITS bit;
};
// ADC External References & Function Declarations:
extern volatile struct ADC_REGS AdcRegs;
```


头文件包含有一个用于器件中每个外设的 .h

DSP2802x_Adc.h

DSP2802x_BootVars.h

DSP2802x_Comp.h

DSP2802x_CpuTimers.h

DSP2802x_DevEmu.h

DSP2802x_Device.h

DSP2802x_ECap.h

DSP2802x_EPwm.h

DSP2802x_Gpio.h

DSP2802x_I2c.h

DSP2802x_NmiIntrupt.h

DSP2802x_PieCtrl.h

DSP2802x_PieVect.h

DSP2802x_Sci.h DSP2802x_Spi.h

DSP2802x_SysCtrl.h

DSP2802x_XIntrupt.h

DSP2802x_Device.h

- ☐ 件 主要的包括头文件
- ☐ 的 将包括所有其他的 .h 文件
- ☐ 在每个源文件中包括此文件： :

```
#include "DSP2802x_Device.h"
```

全局变量定义文件

DSP2802x_GlobalVariableDefs.c

- 定义每个外设寄存器结构体变量
- 每个结构体使用一个 `DATA_SECTION` 宏指令来强制结构体的存储位置，以实现至正确的地址链接

DSP2802x_GlobalVariableDefs.c

```
#include "DSP2802x_Device.h"
...
#pragma DATA_SECTION(AdcRegs,"AdcRegsFile");
volatile struct ADC_REGS AdcRegs;
...
```

- ◆ 将每个结构链体接至采用结构命名代码段的外设映射存储地址

DSP2802x_Headers_nonBIOS.cmd

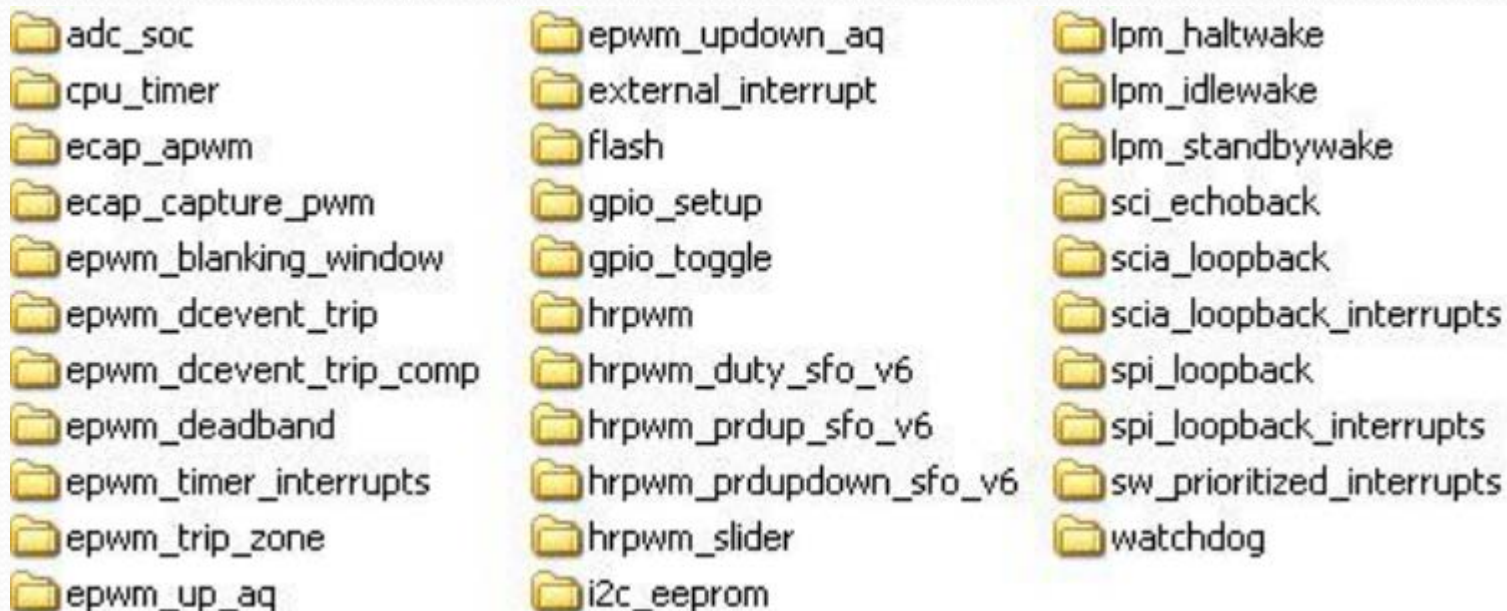
```
MEMORY
{
    PAGE1:
        ...
        ADC:  origin=0x007100, length=0x000080
        ...
}

SECTIONS
{
    ...
    AdcRegsFile:  > ADC      PAGE = 1
    ...
}
```

- ◆ 将该文件添加至您的 **CCS** 项目:

DSP2802x_nonBIOS.cmd

- 针对每个外设应用的项目范例
- 可帮助使用者快速启动开发工作



●4. 2 TMS320F28027最小系统简介

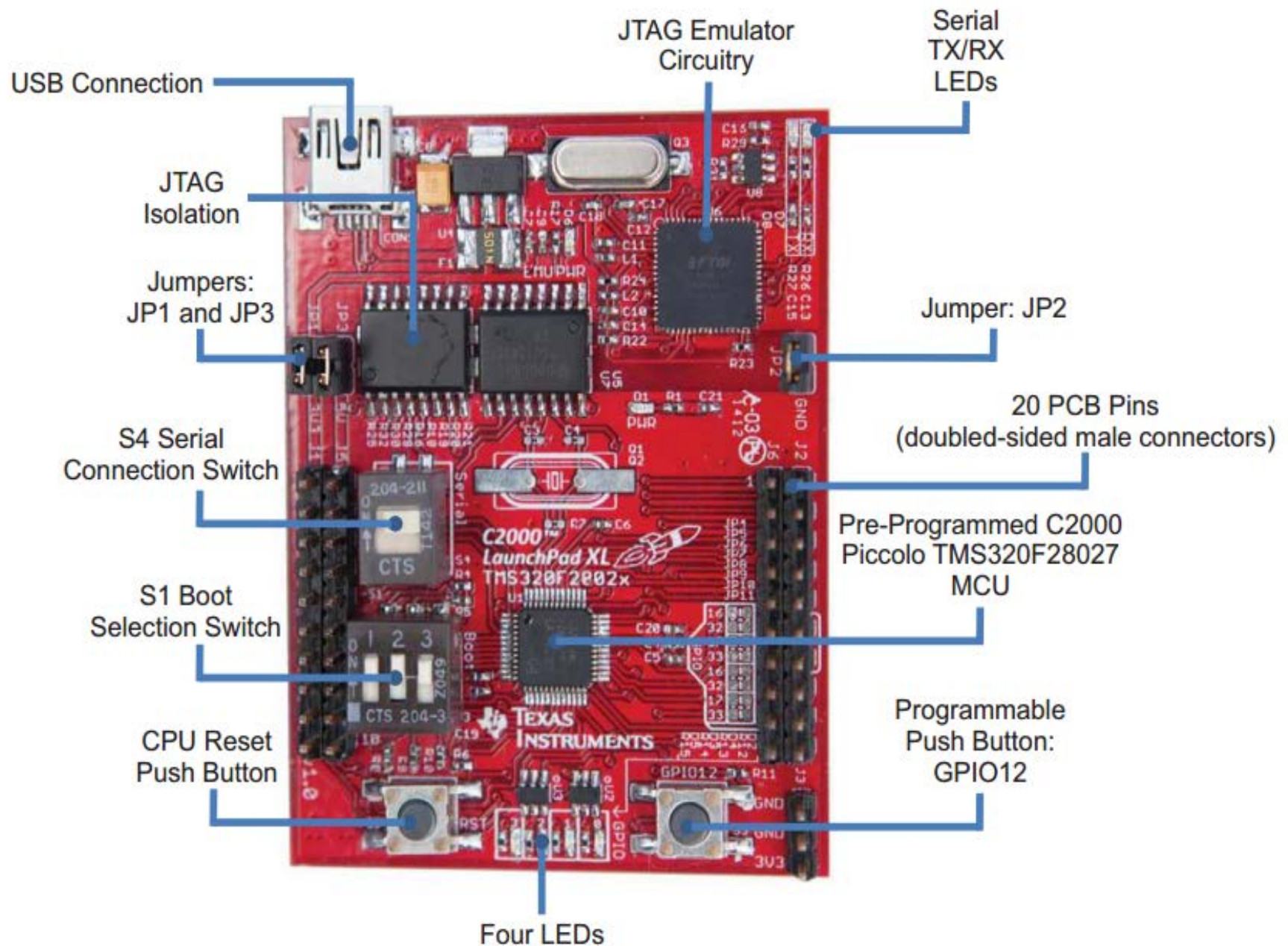
以TI公司提供的C2000实验套件**LAUNCHXL-F28027**为例进行介绍F28027的最小系统：

实验套件包括两部分：

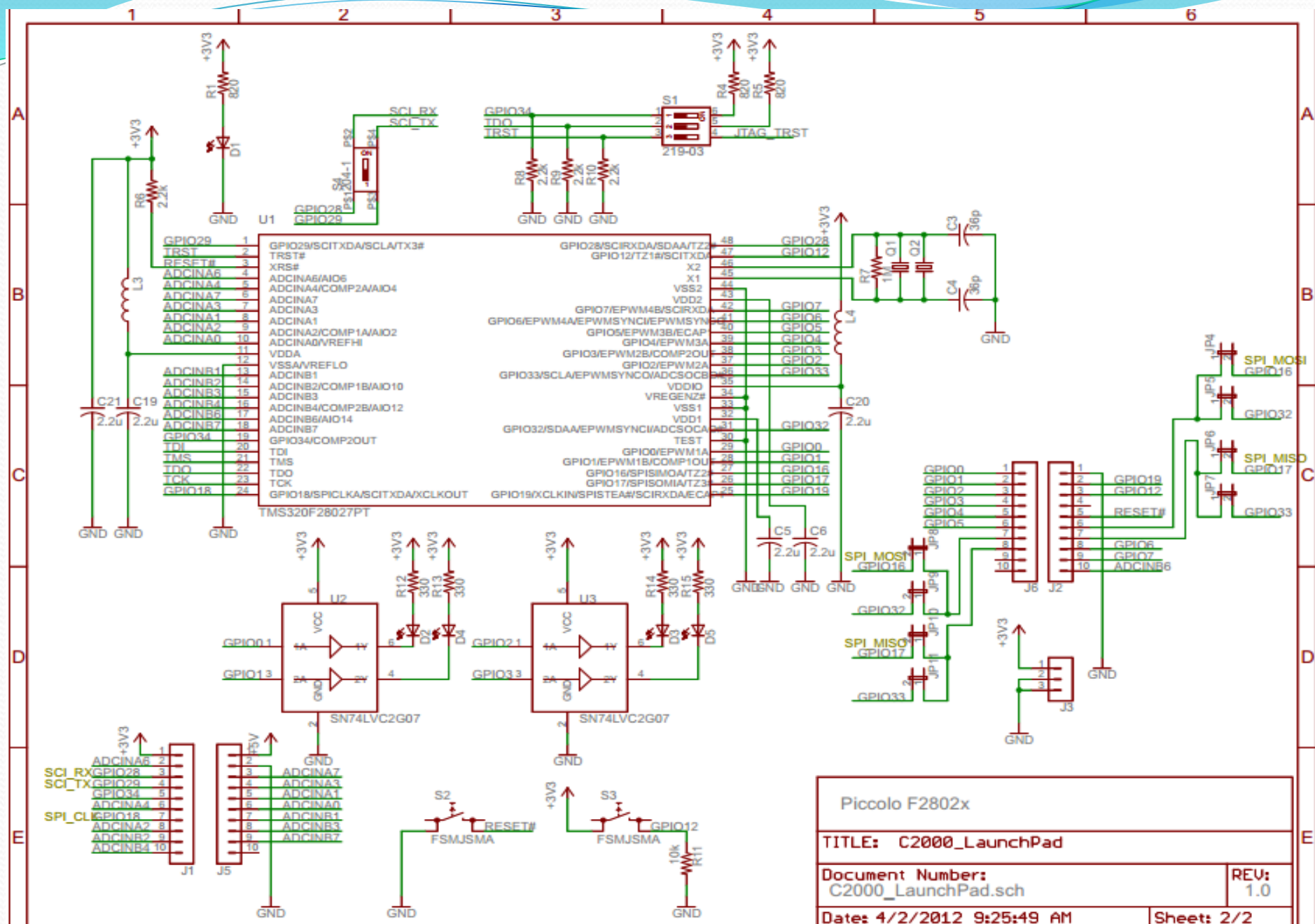
XDS100V2仿真器，通过USB和电脑连接；

F28027的最小系统，最小系统包括DSP的**CPU**、**BOOT选择**、**复位电路**、**时钟电路**、**简单IO电路**（4个LED灯、一个可编程按键）以及两个芯片引脚引出座子。

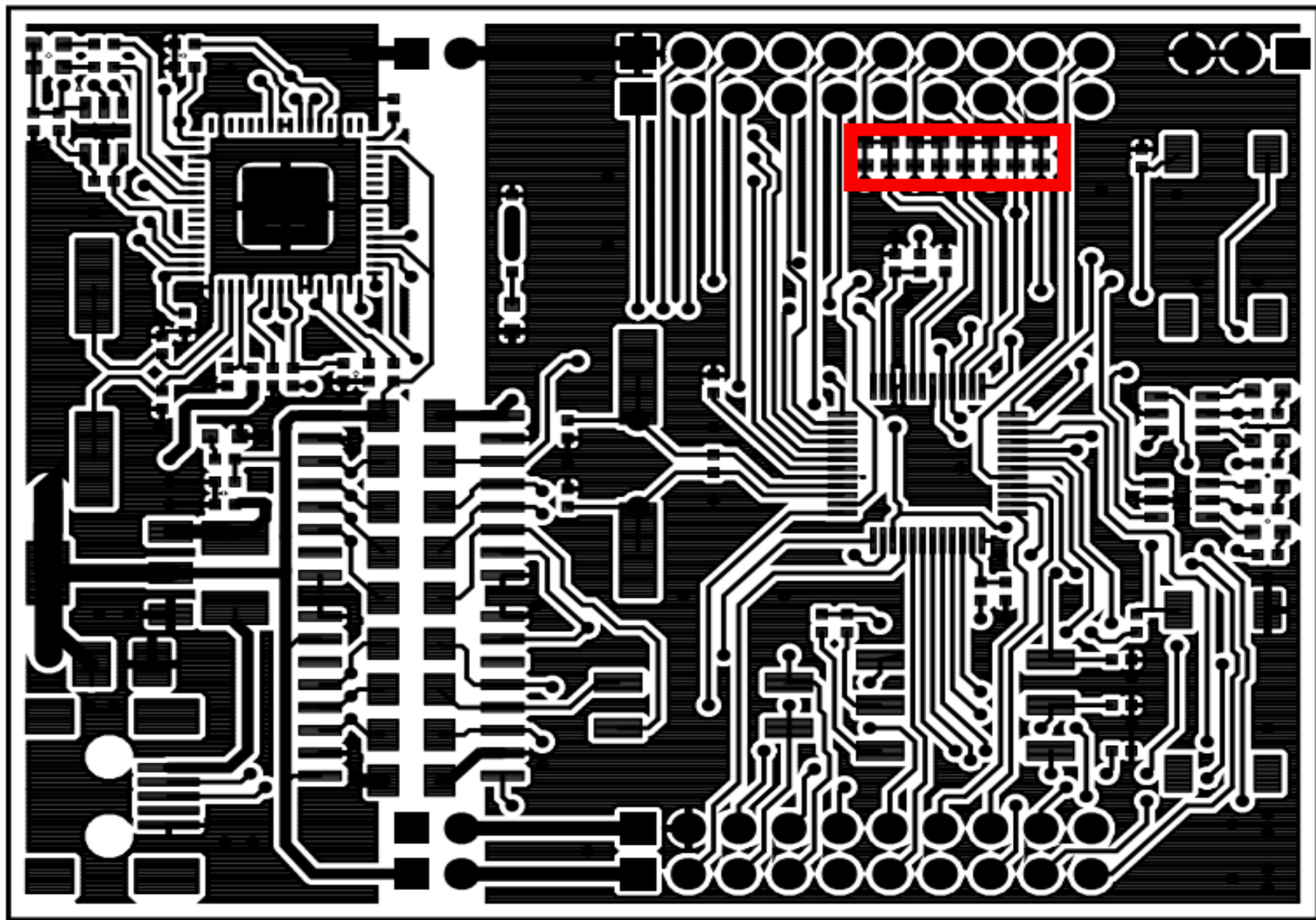
TI 公司提供的最小系统



最小系统原理图



最小系统PCB正面映射



最小系统供电方式

Power Domain

The C2000 LaunchPad has two separate power domains for the purpose of allowing JTAG isolation. Jumpers JP1, JP2, and JP3 configure whether the USB power is passed to the target device.

Jumper	Power Domain
JP1	3.3 V
JP2	Ground
JP3	5 V

最小系统启动模式选择

Boot Mode Selection

The LaunchPad's F28027 device includes a boot ROM that performs some basic start-up checks and allows for the device to boot in many different ways. Most users will either want to perform an emulation boot or a boot to flash (if they are running the application standalone). S1 has been provided to allow users to easily configure the pins that the bootROM checks to make this decision. The switches on S1 correspond to:

Switch	Function
1	GPIO34
2	GPIO37
3	TRSTn

Keep in mind that the debugger does not connect if the device is not in the emulation boot mode (TRST switch in the up position). More information about boot mode selection can be found in the *TMS320x2802x Piccolo Boot ROM Reference Guide* ([SPRUFN6](#)).

最小系统引脚分配

Table 1. C2000 LaunchPad Pin Out and Pin Mux Options

Mux Value						Mux Value			
3	2	1	0	J1 Pin	J5 Pin	0	1	2	3
			+3.3V	1	1	+5V			
			ADCINA6	2	2	GND			
TZ2	SDAA	SCIRXDA	GPIO28	3	3	ADCINA7			
TZ3	SCLA	SCITXDA	GPIO29	4	4	ADCINA3			
Rsvd	Rsvd	COMP2OUT	GPIO34	5	5	ADCINA1			
			ADCINA4	6	6	ADCINA0			
	SCITXDA	SPICLK	GPIO18	7	7	ADCINB1			
			ADCINA2	8	8	ADCINB3			
			ADCINB2	9	9	ADCINB7			
			ADCINB4	10	10	NC			
3	2	1	0	J6 Pin	J2 Pin	0	1	2	3
Rsvd	Rsvd	EPWM1A	GPIO0	1	1	GND			
COMP1OUT	Rsvd	EPWM1B	GPIO1	2	2	GPIO19	SPISTEA	SCIRXDA	ECAP1
Rsvd	Rsvd	EPWM2A	GPIO2	3	3	GPIO12	TZ1	SCITXDA	Rsvd
COMP2OUT	Rsvd	EPWM2B	GPIO3	4	4	NC			
Rsvd	Rsvd	EPWM3A	GPIO4	5	5	RESET#			
ECAP1	Rsvd	EPWM3B	GPIO5	6	6	GPIO16/32	SPISIMOA/ SDAA	Rsvd/ EPWMSYNCI	TZ2/ ADCSOCA
TZ2/ ADCSOCA	Rsvd/ EPWMSYNCI	SPISIMOA/ SDAA	GPIO16/32	7	7	GPIO17/33	SPISOMIA/ SCLA	Rsvd/ EPWMSYNCO	TZ3/ ADCSOCB
TZ3/ ADCSOCB	Rsvd/ EPWMSYNCO	SPISOMIA/ SCLA	GPIO17/33	8	8	GPIO6	EPWM4A	EPWMSYNCI	EPWMSYNCO
			NC	9	9	GPIO7	EPWM4B	SCIRXDA	Rsvd
			NC	10	10	ADCINB6			

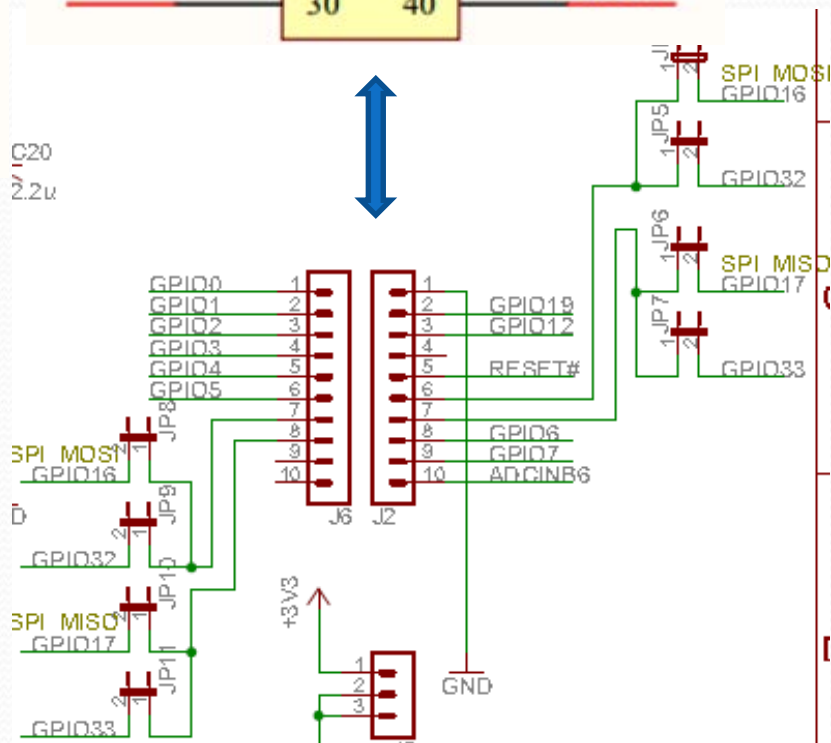
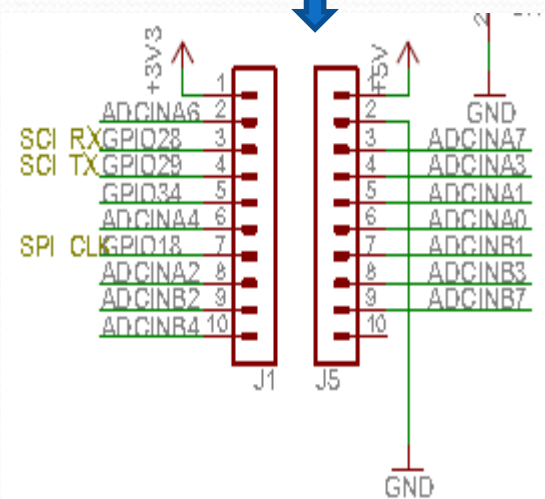
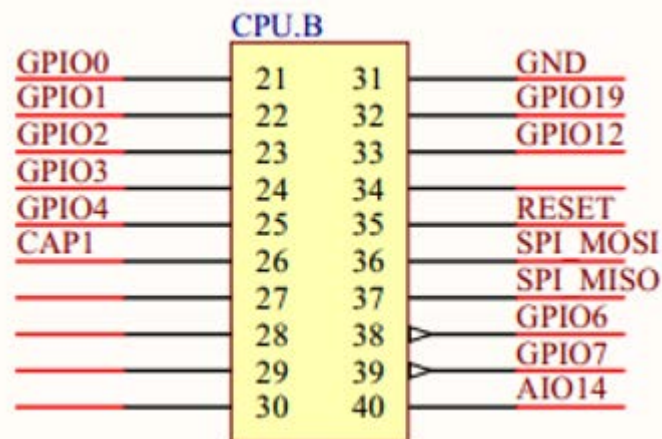
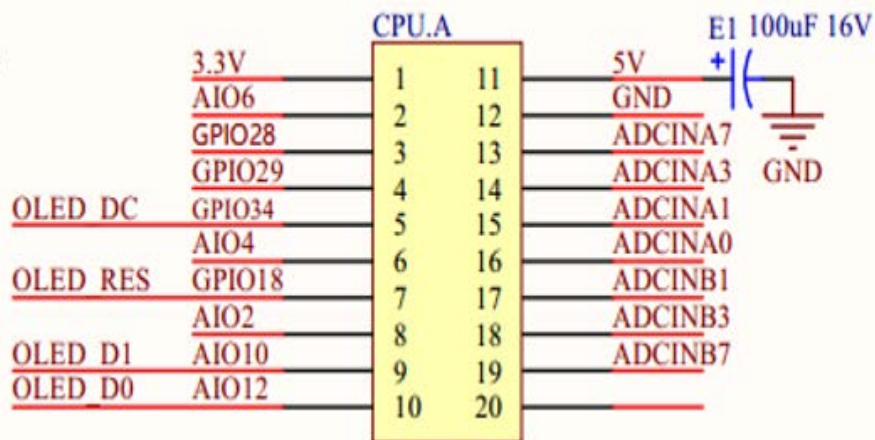
4. 3实验套件应用扩展示例

虽然通过TI公司的实验套件LAUNCHXL-F28027可进行TMS320F28027的初步实验，包括I/O口实验、定时器实验、捕获实验、PWM实验等等，

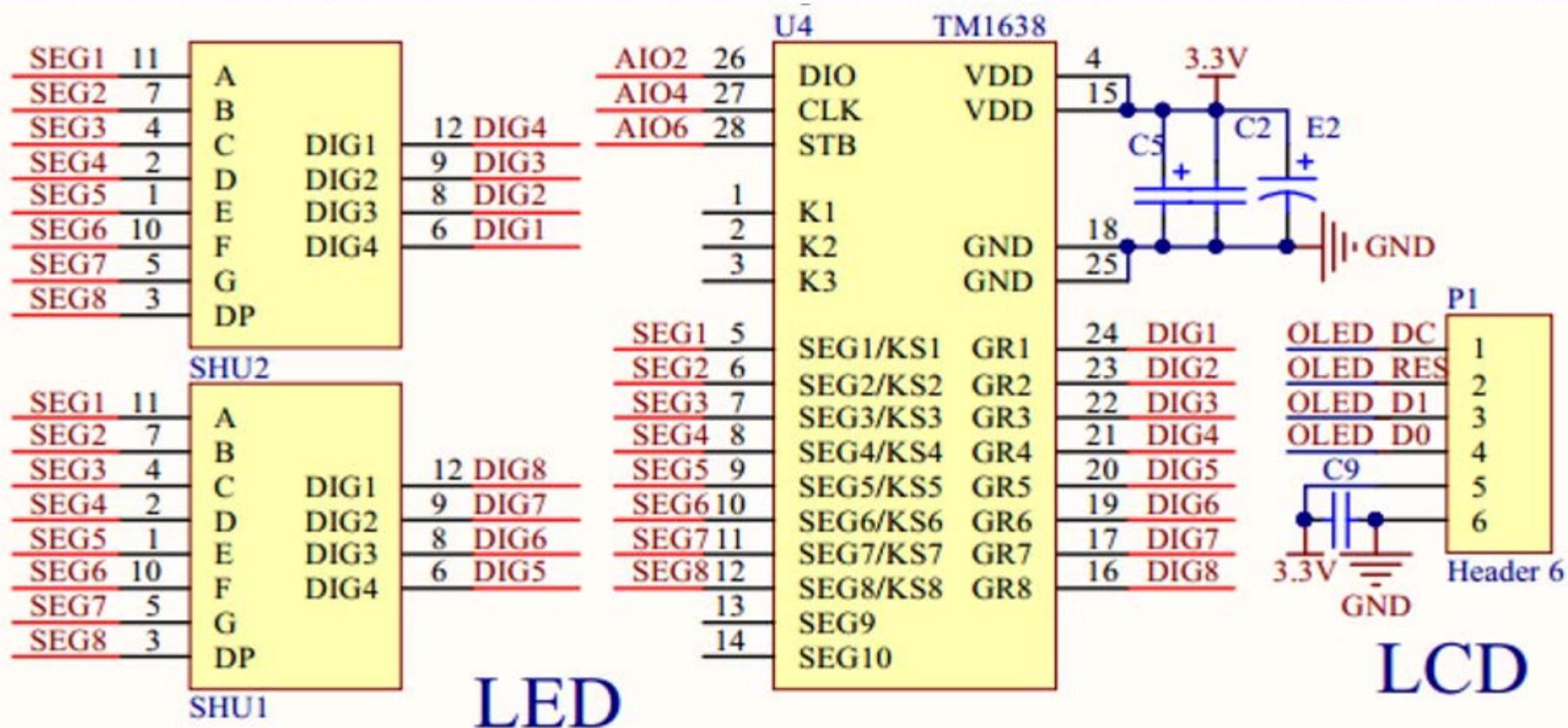
但是直接利用该套件进行实验一是交互性差，二是扩展脚上直接操作既不方便又容易出错，甚至容易短路烧毁套件。

一种可行的方法是在套件基础上做应用扩展，把该套件看作一个带仿真器的CPU。

与最小系统板接口



人机交互接口



人机交互的扩展

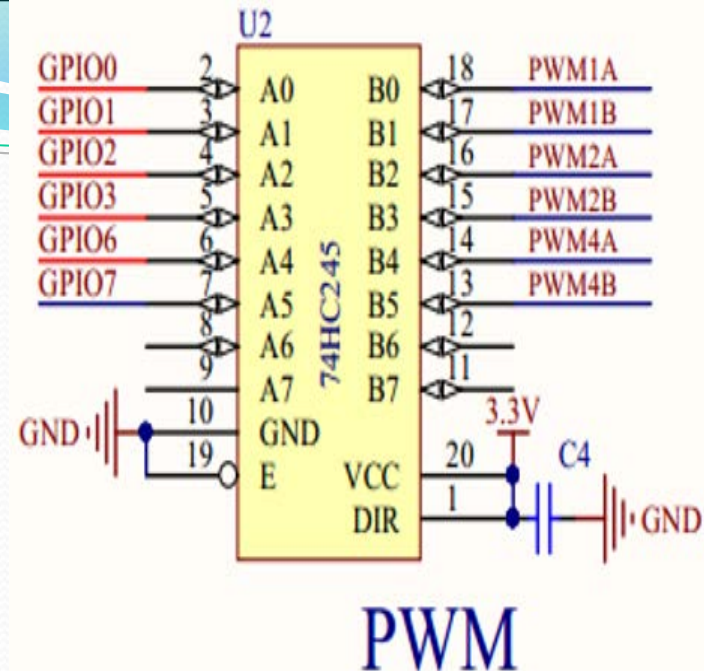
通过AIO2、AIO4、AIO6利用TM1638扩展了8位LED数码管显示。

也可通过AIO10、AIO12、GPIO18、GPIO34扩展一个OLED显示。

这两个显示扩展既强化了IO实验，又增强了人机交互，增加了实验装置直观性。

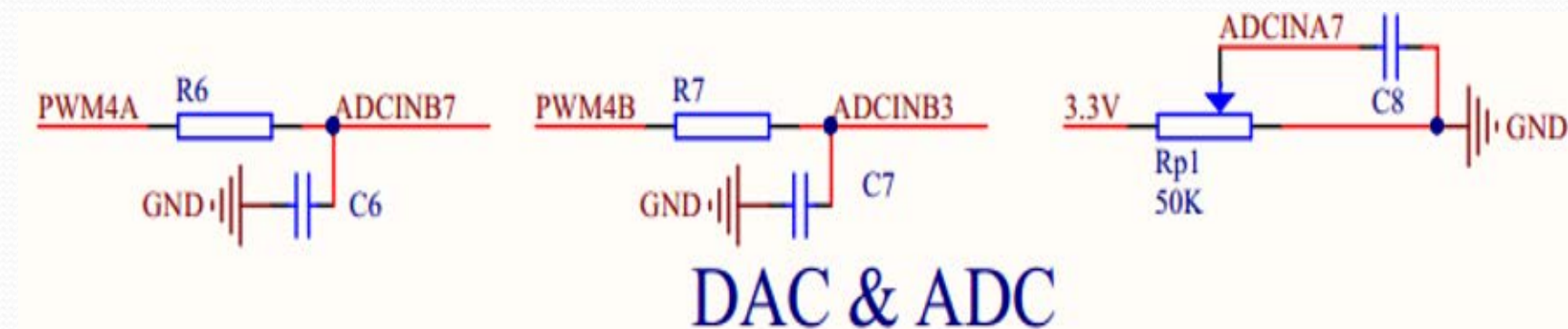
ePWM应用扩展

PWM是DSP非常重要的控制外设。
扩展一片74HC245扩展为ePWM1A/B、
ePWM2A/B和ePWM4A/B，可进行三相
互补PWM应用。



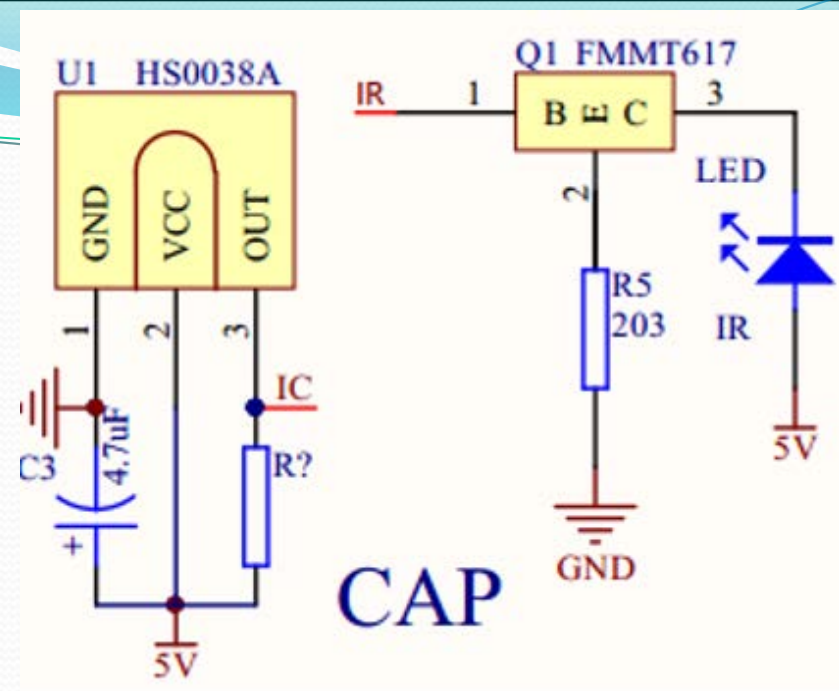
DAC/ADC应用扩展

通过对ePWM4A/B进行低通滤波，PWM就用作了DAC



eCAP应用扩展

可以将PWM信号和CAP1引脚相连
也可以捕获红外接收器的输出

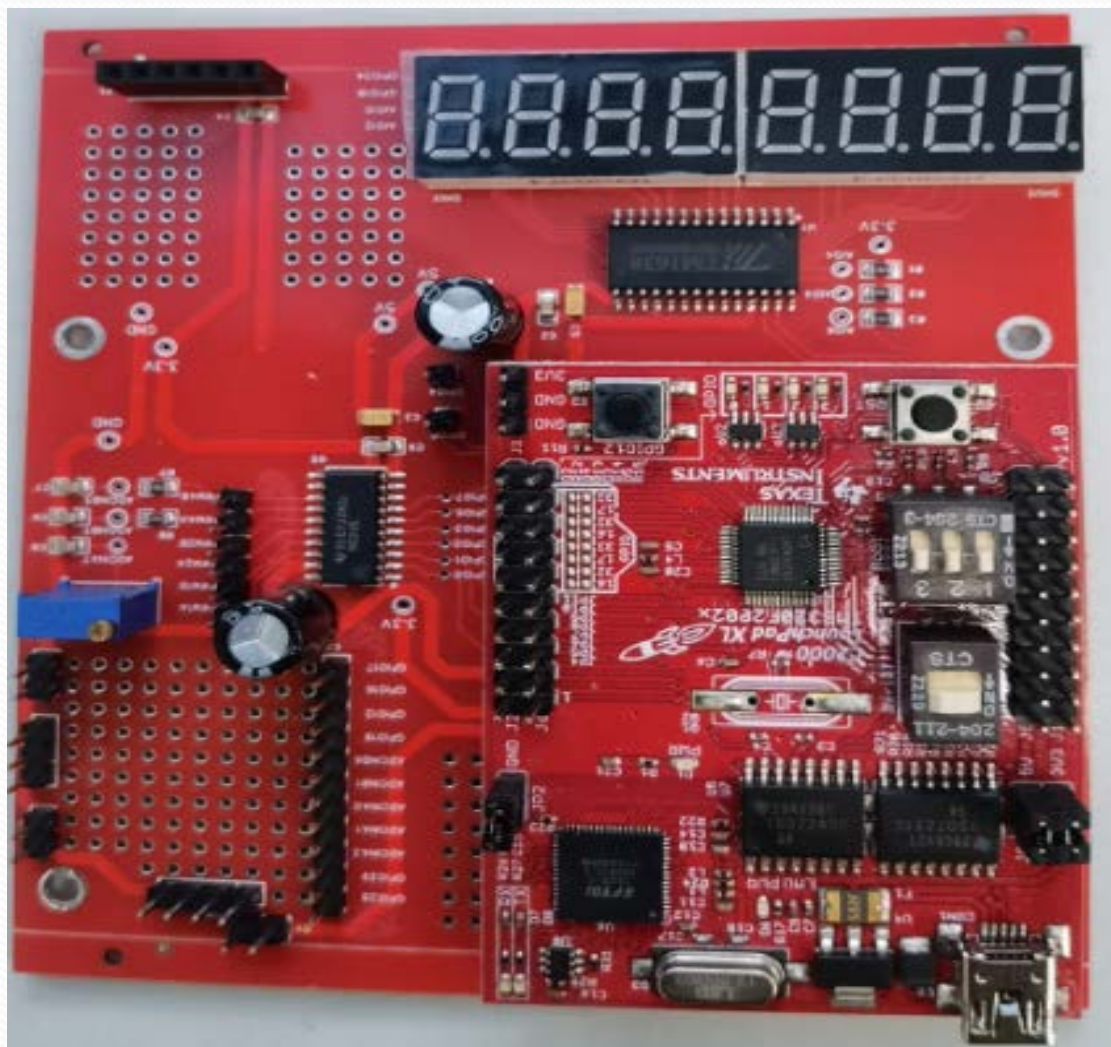


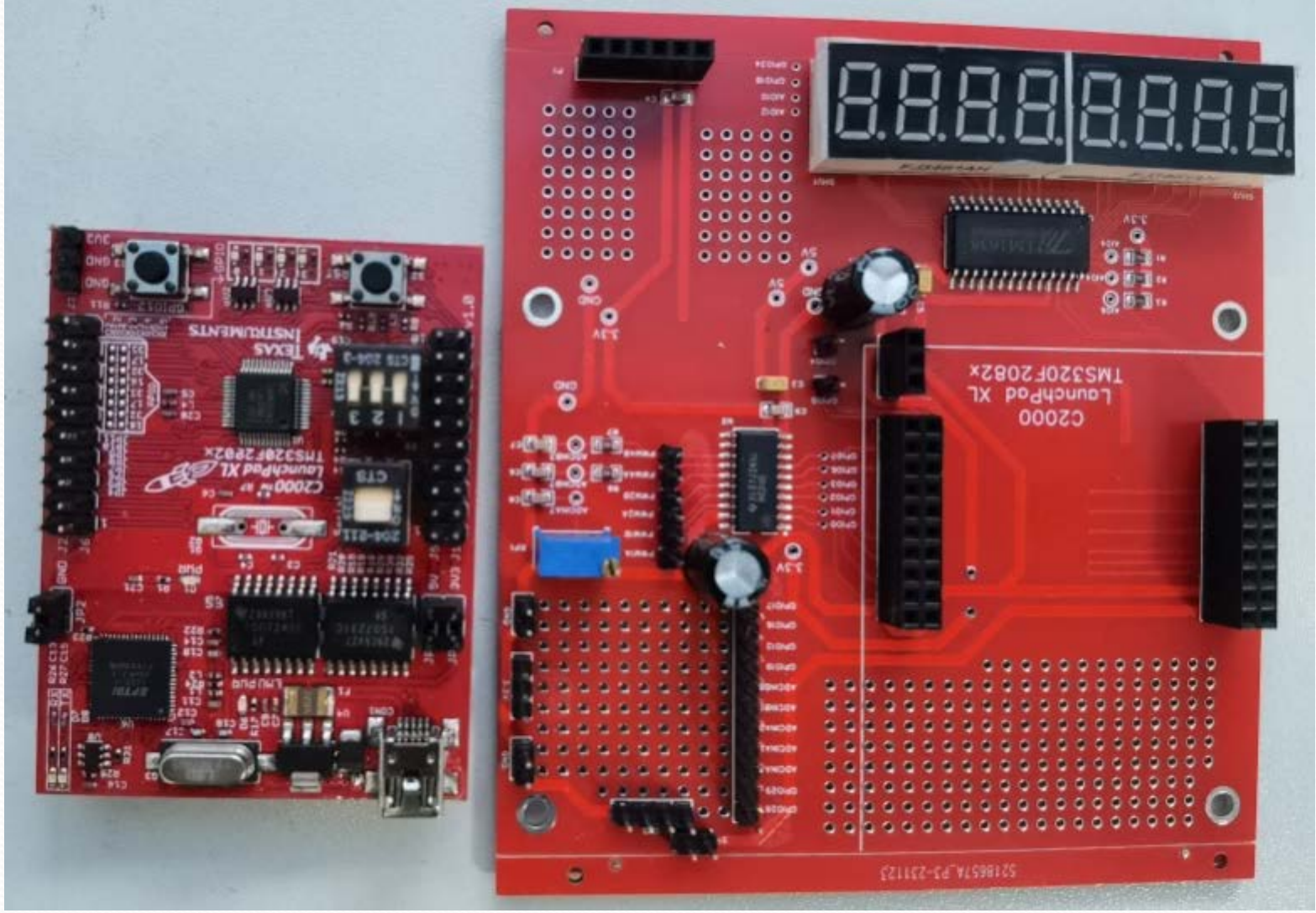
综合应用

装置中GPIO12和ADCINA7可用作指令和输入，
GPIO0~3和GPIO6~7（ePWM）等可作为输出，
加上串行通信接口和红外输出接口，
再加上LED和OLED显示等，

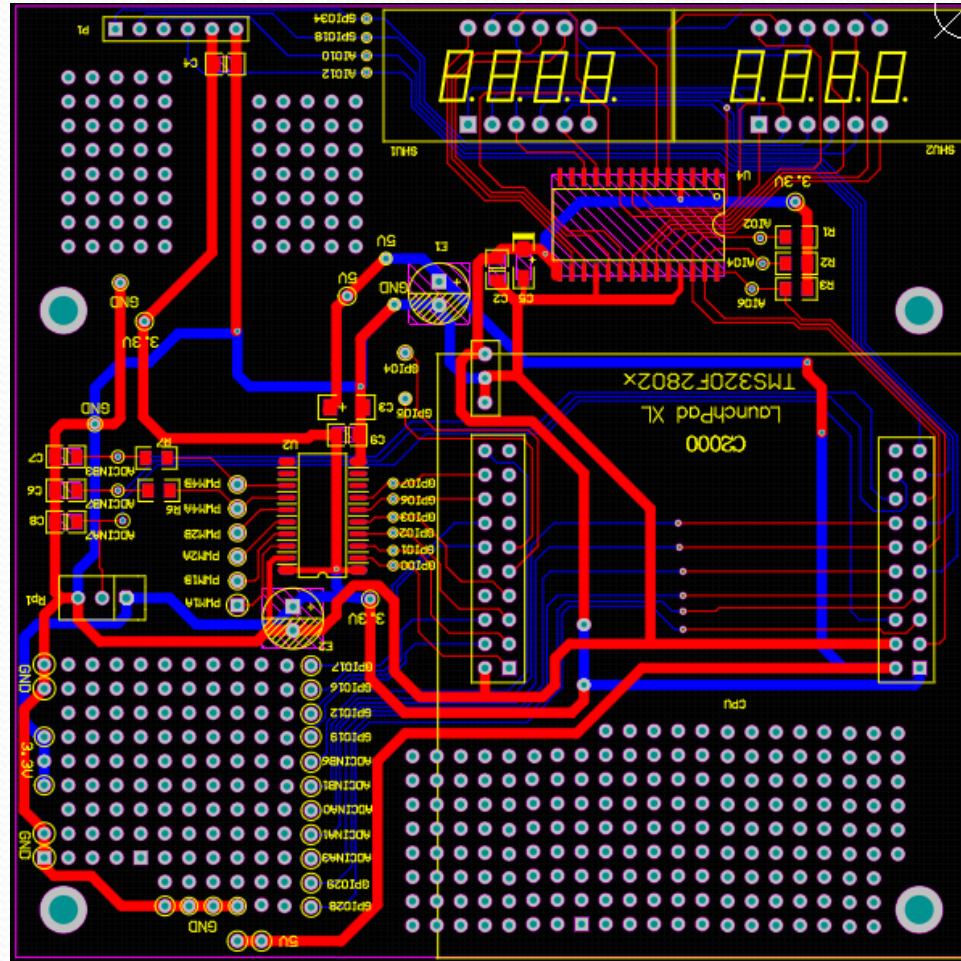
可构成一个相对完整实用的系统

4.3.2 DSP实验设备介绍

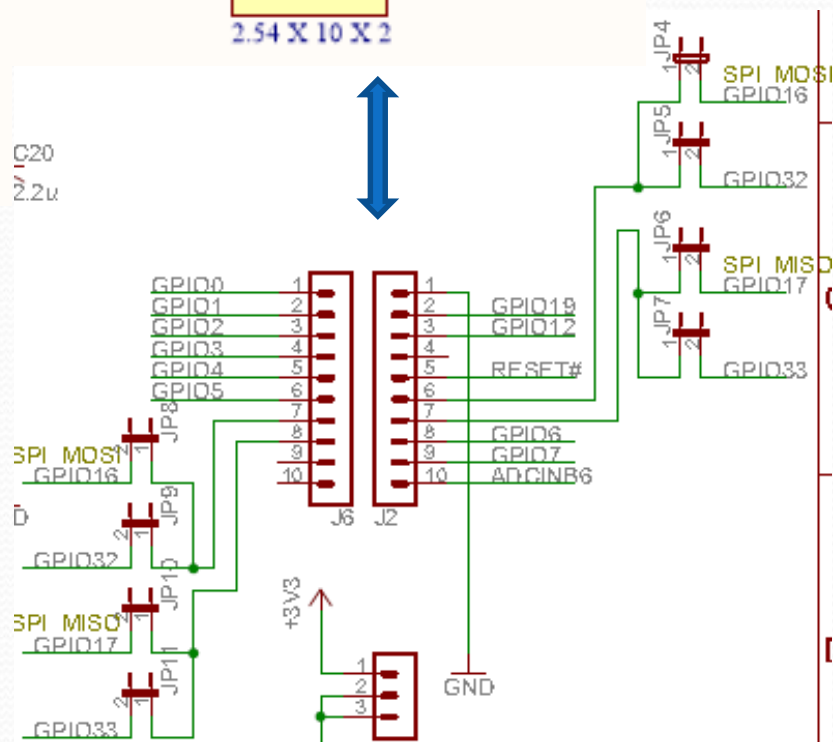
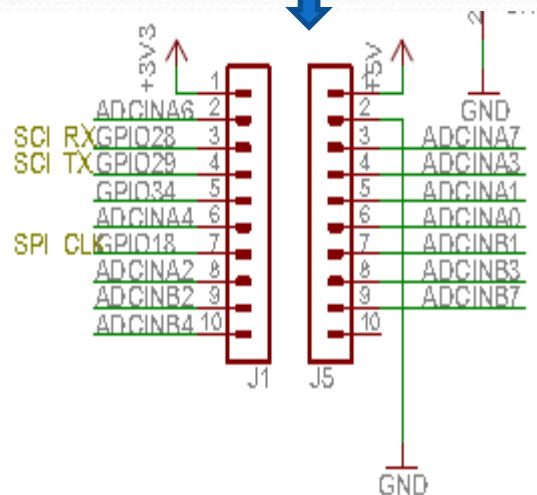
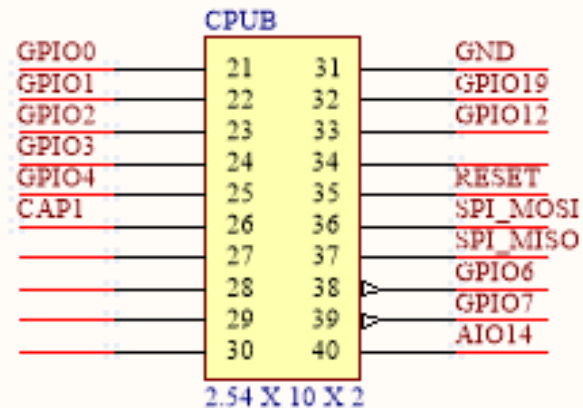
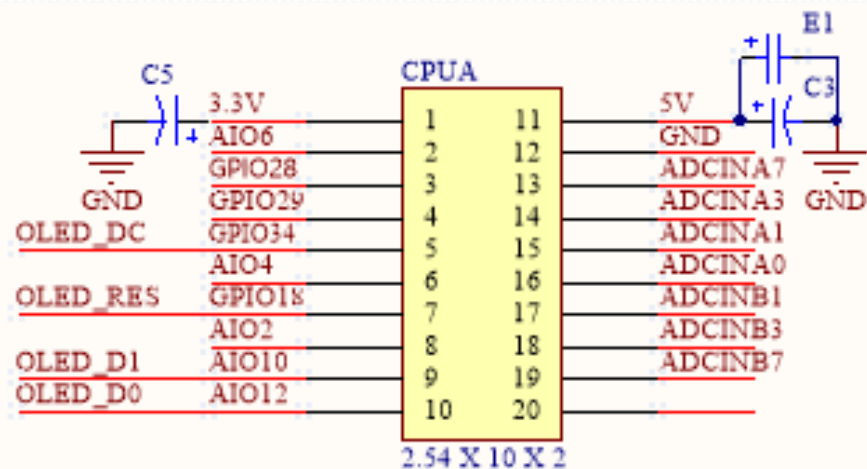




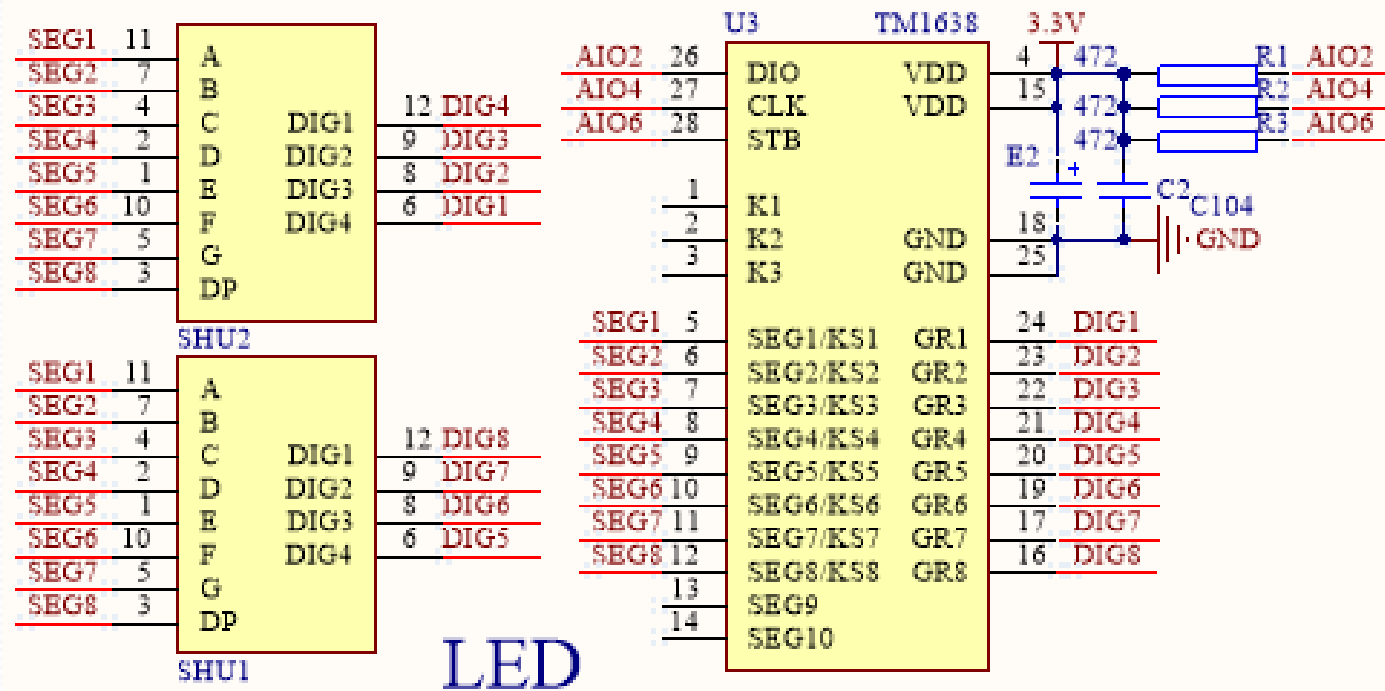
实验板之目标板PCB图



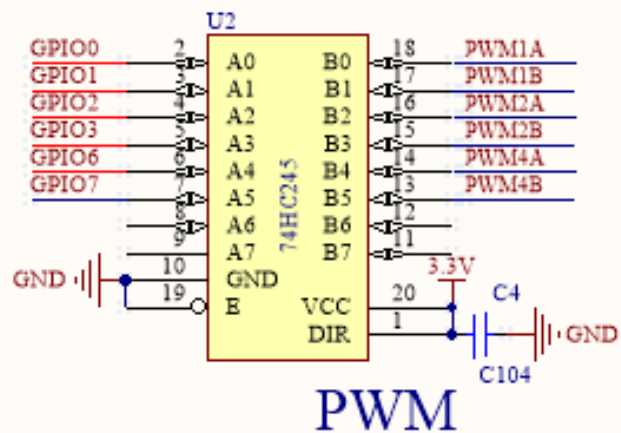
与最小系统板接口



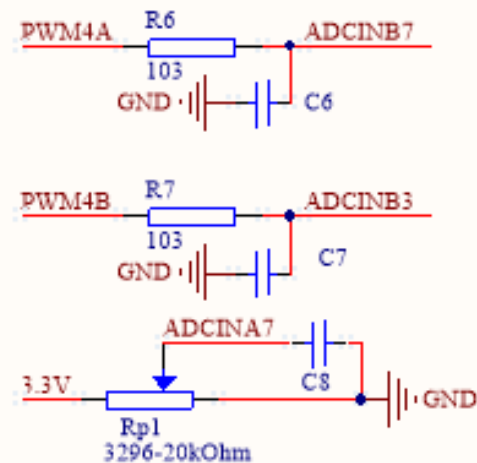
LED动态扫描显示部分电路



PWM、DAC/ADC电路



DAC & ADC



●4.4 集成开发环境CCS

●4.4.1 CCS简介

CCS (Code Composer Studio) 是业界领先的集成开发环境 (IDE)，包含一套用于开发、编译、调试和分析嵌入式应用的工具。

它基于 Eclipse 开源开发环境，因此可与大量工具集成。

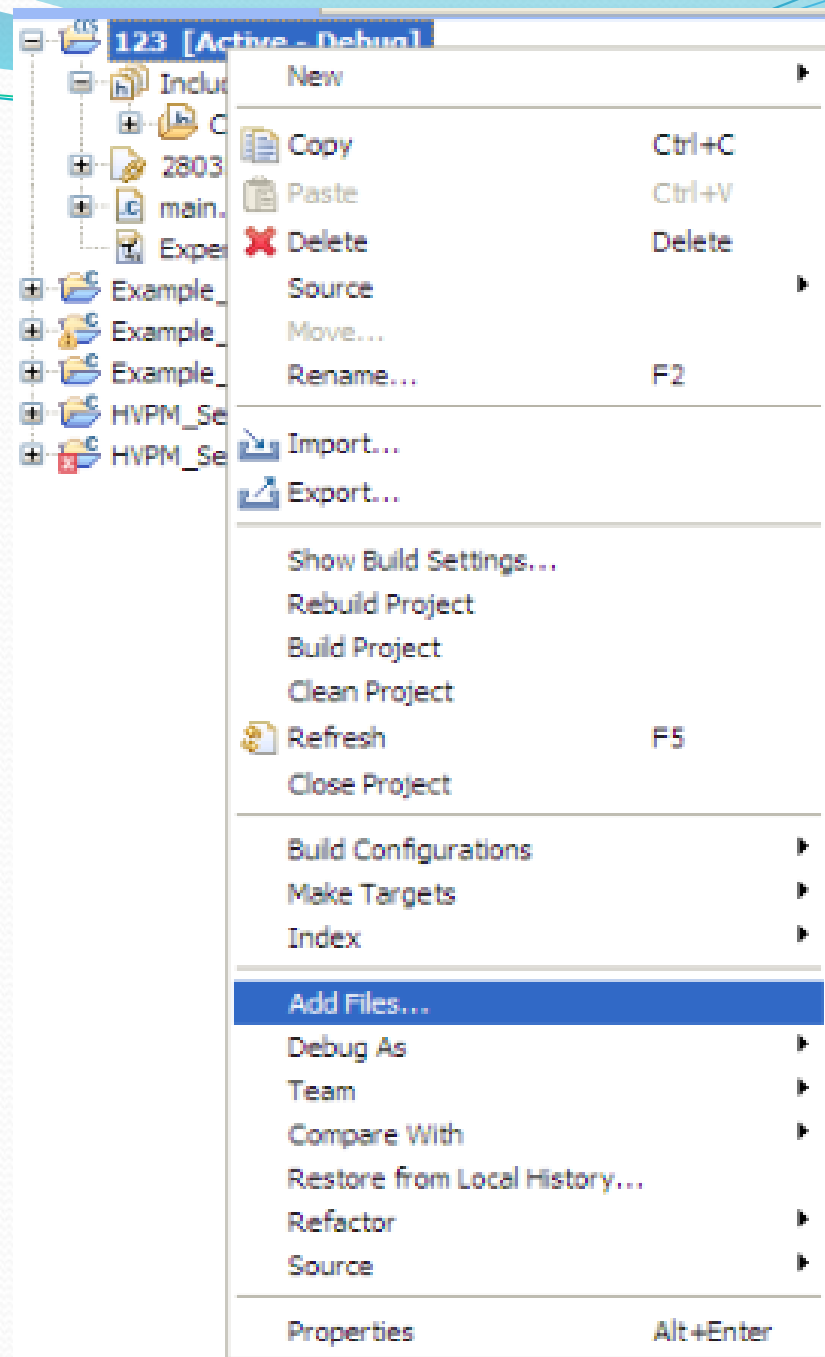
CCS包含优化型 C/C++ 编译器、源代码编辑器、项目构建环境、调试器、性能评测工具以及多种其他功能。

CCS提供单一用户界面，可帮助完成应用开发流程的每个步骤。

它的工具和界面是之前较为熟悉的形式，可以快速入手。

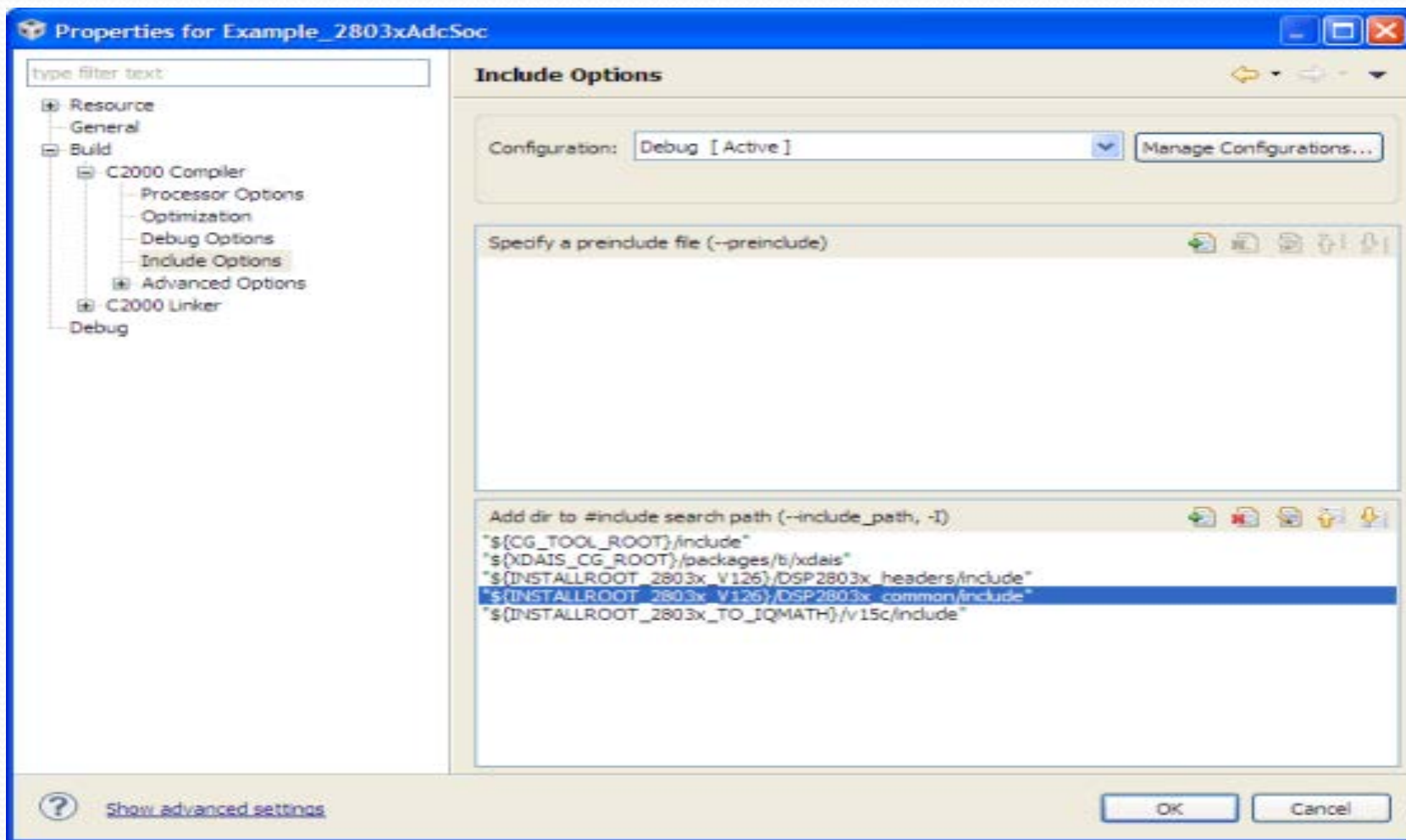
●4.4.2 新建工程

添加现有源文件到
工程中。右击工程
名-> Add Files...



添加头文件目录:

一右击工程名->Properties,
选择 Build->Include Options,
有个#include 搜索路径, 添加上路径即可。

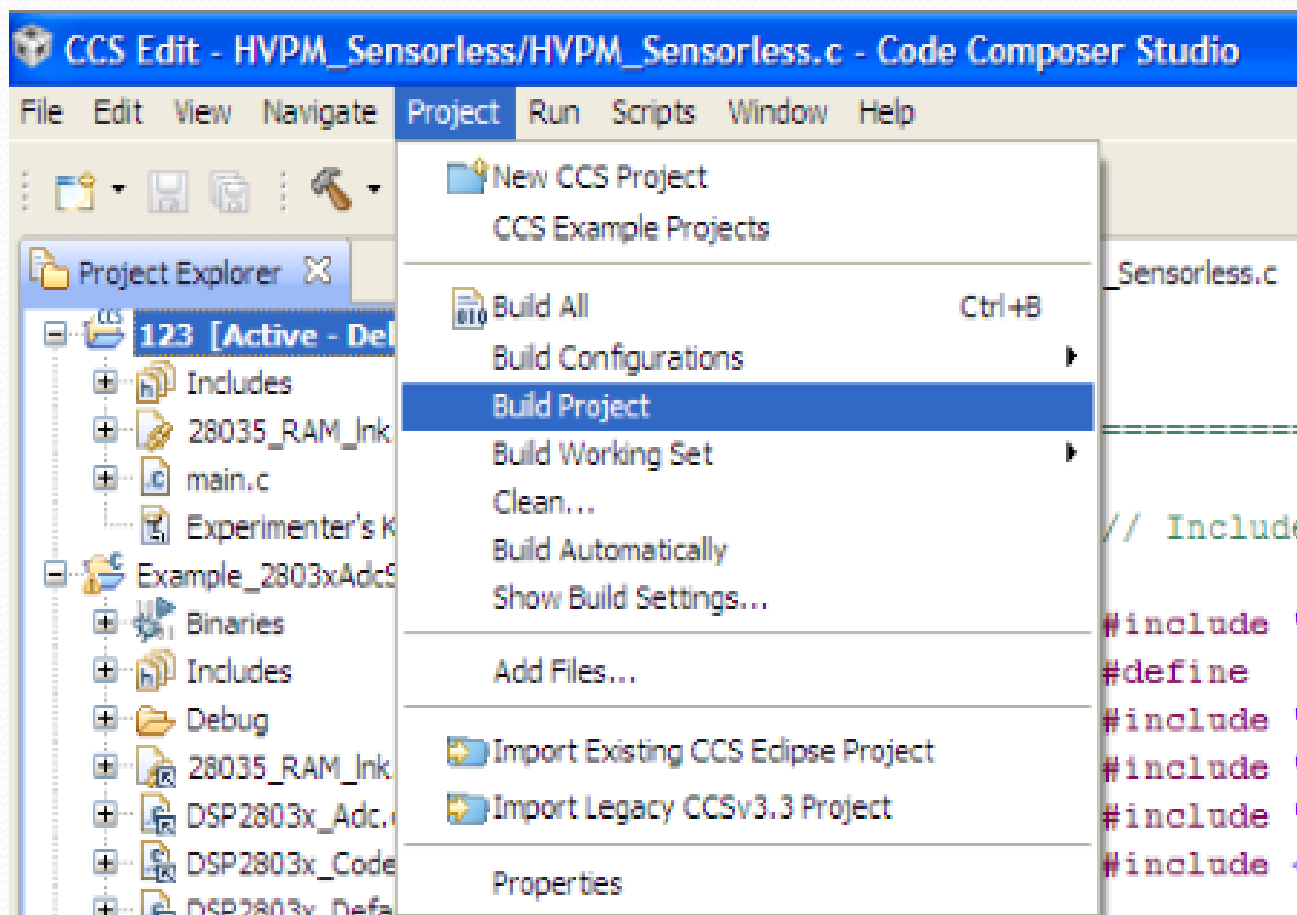


最有效的方法：

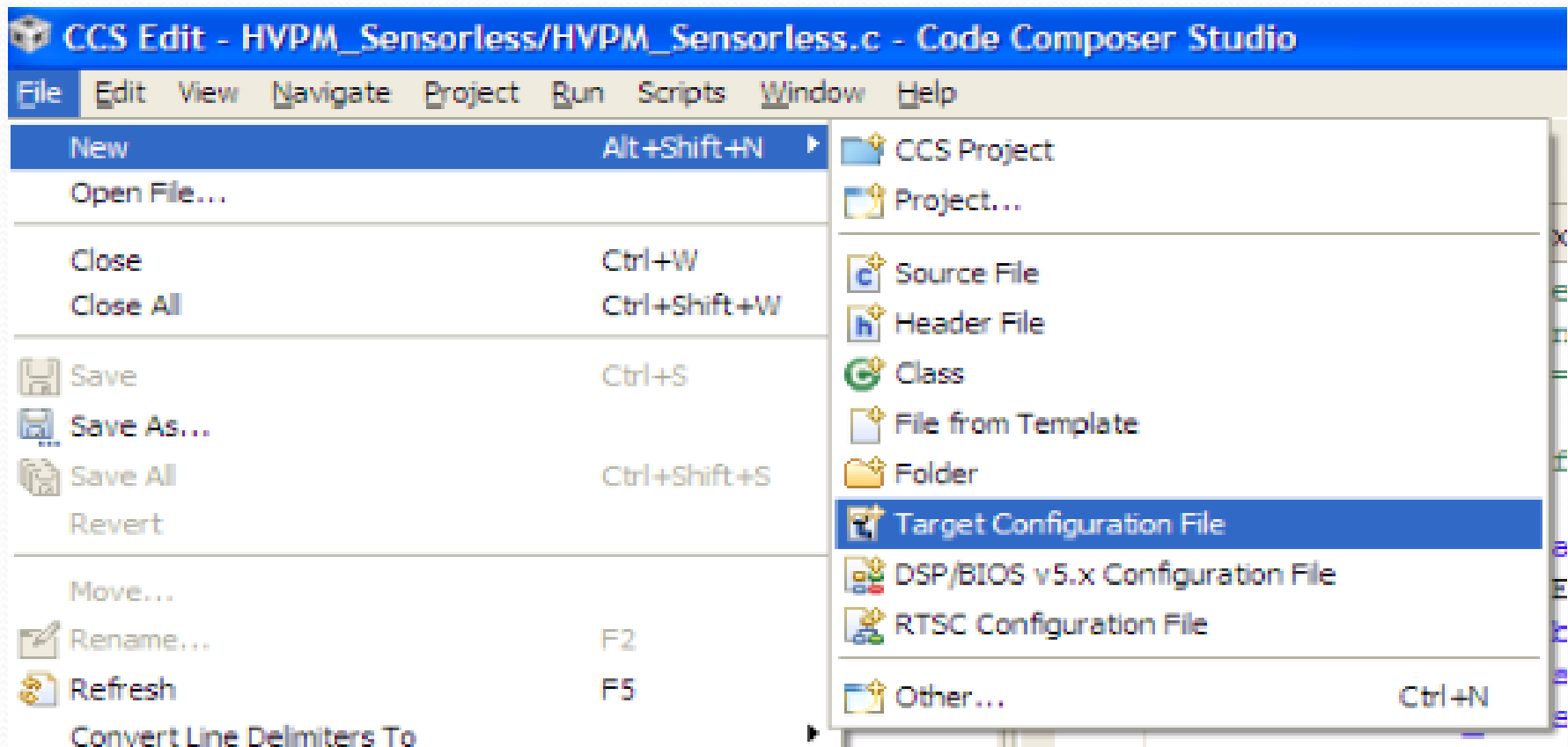
**拷贝一个现有的工程
然后加以裁剪和添加**

4.4.3 CCS工程调试

编译工程 选择 Project->Build project



在 Debug 之前，一定要选择和设置目标配置文件（Target Configurations）。
新建目标配置文件（Target Configuration File），如下图所示。



如下图所示，选择仿真器，选择平台，完成后可以点击 Test Connection，进行仿真器连接测试，如果成功，说明硬件没有问题。

Basic

General Setup

This section describes the general configuration about the target.

Connection

Board or Device

- ☐ TMS320F280270
- ☐ TMS320F28030
- ☐ TMS320F28031
- ☐ TMS320F28032
- ☐ TMS320F28033
- ☐ TMS320F28034
- ☒ TMS320F28035
- ☐ TMS320F28044
- ☐ TMS320F28050
- ☐ TMS320F28051

Note: Support for more devices may be available from the update manager.

Advanced Setup

Target Configuration: lists the configuration options for the target.

Save Configuration

Test Connection

To test a connection, all changes must have been saved, the configuration file contains no errors and the connection type supports function.

Debug

在Test Connection成功后，就可以进行Debug:

跟编译工程类似的方法进行工程的 Debug

最简单的方式就是点击如下图所示的小虫子。

